

Spiking Neural Networks on GPUs

Evaluating four frameworks across four event-driven computer-vision applications — objective, reasoning, and the experimental setup

The story

An ordinary camera takes a complete picture of everything in view, then another, then another, thirty or sixty times a second. Every pixel reports its brightness every time, whether anything changed or not. Point one at a parked car and it faithfully records the same parked car eighteen hundred times a minute.

An **event camera** works differently. It has no shutter and takes no pictures. Each pixel watches its own small patch of the world and stays silent until the brightness it sees changes. The moment it does, that pixel alone sends one short message: where it is, when it happened, and whether the light went up or down. Nothing moves, nothing is sent. Something moves, and only the pixels along the moving edge speak up — within microseconds rather than at the next frame tick. The result is not a stack of pictures but a stream of changes.

A **spiking neural network** is built on the same principle. Its artificial neurons stay quiet until enough evidence arrives, then emit a single brief pulse — a spike — and fall quiet again. Work happens only where something arrived. A conventional network, by contrast, computes every value in every layer on every pass, whether or not there was anything to say.

So the two are not merely thematically similar. They share one physical property: **both are sparse, and both act only when something happens**. The camera's output format is already the network's native input format, which means the sparsity survives from photon to answer instead of being manufactured at one end or discarded at the other. That is the claim this experiment exists to test, and it is the reason the work is built on event-driven applications rather than ordinary video.

Objective

Determine whether spiking networks offer a **measurable** advantage over an equivalent non-spiking network on event-camera vision, and whether that advantage grows with how much the task depends on time — using four independent spiking frameworks, so that no conclusion rests on any single library's implementation.

Why this is the right question

There are four ways to pair a camera type with a network type, and only one of them keeps the sparsity intact. An ordinary network fed event data must first pad all that silence back into full frames — paying for a sensor that ignores stillness, then re-inventing the stillness. A spiking network fed ordinary video must invent spikes out of brightness values, manufacturing a sparsity the data never had. Event camera with spiking network is the one arrangement where nothing is faked and nothing is thrown away.

Why train on GPUs, when the appeal of spiking networks is neuromorphic hardware? Neuromorphic chips run a *trained* spiking network extremely cheaply, but they are poor places to *teach* one: limited memory, not designed for the heavy repeated passes that training demands, and largely unavailable in practice. GPUs are the opposite — plentiful, memory-rich, and supported by every training tool that exists. The honest division of labour is therefore to train on a GPU, because that is where training is possible, and to estimate what the trained network would cost on the hardware it is ultimately intended for. This is why two energy figures are reported throughout and never merged: the measured GPU energy says what this experiment actually cost, and the synaptic-operation estimate says what the same network would cost where it belongs.

The evaluation grid

A spiking network and a conventional one cannot be compared on spike counts, because only one of them has spikes. Five measures are therefore defined so that each means exactly the same thing for both, and all five are recorded for every run.

Measure	Definition, identical for both model types	Answers
Task score	Accuracy for the classification applications; root-mean-square error for pose; average end-point error for optical flow. Same test set, same definition.	Does it work?
Activation density	The fraction of units that emitted anything, measured layer by layer. For a spiking layer this is its spike rate; for a conventional layer it is the fraction of units above zero. One minus this figure is the sparsity on which the whole efficiency argument rests.	How much of the network did nothing?
Synaptic operations	Operations required for one inference — accumulations for the spiking model, multiply-accumulations for the conventional one — counted the same way from the same architecture, together with the energy each operation type costs.	What would this cost on the right hardware?
Measured GPU energy	Joules drawn during the run, reported both as a total and as the dynamic portion above the idle baseline, sampled directly from the hardware.	What did it cost here and now?
Single-stream latency	Time from one sample arriving to its answer being ready, at batch size one, reported as median and 90th percentile from a dedicated untimed pass.	How quickly does one answer arrive?

Recorded alongside these as supporting context: per-layer spike counts, inter-spike-interval variability, peak and reserved GPU memory, throughput, amortised per-sample latency, GPU temperature and clock speeds, parameter counts, and the credit-assignment algorithm each model trains with.

Why four applications

The four datasets are not four opportunities to demonstrate the same thing. They are four points along a single axis: **how much the task depends on time**. A spiking neuron carries its internal state from one moment to the next and can therefore accumulate evidence across a sequence; a conventional unit has no memory and treats every moment independently. If that difference is worth anything, it should become more valuable as the task depends more on time.

Application	Dependence on time	What it contributes
N-Caltech101 object recognition	Near zero. The objects are stationary; events exist only because the camera was swept across them.	The control end of the axis. A spiking advantage here would be the surprise.
DVS128 Gesture hand-gesture recognition	High. The answer is a movement — no single moment distinguishes a clockwise from a counter-clockwise gesture.	The direct counterpart to the control. This is where memory across time should pay.
DAVIS Camera Pose 6-DOF trajectory	Continuous. The camera's position and orientation change smoothly and must be tracked.	Leaves classification behind: a different answer at every moment, not one at the end.
DSEC optical flow	Both spatial and temporal. A motion vector for every pixel, and motion exists only across time.	The most demanding case, and the one an event camera is least replaceable for.

The claim these four test together: **the spiking model's advantage over its non-spiking equivalent should widen as dependence on time increases**. If it does, the experiment has shown what spiking is specifically good for. If it does not — if the conventional model keeps pace even on gesture and flow — that is a stronger and considerably rarer finding, and it is reported just as plainly.

What has been built

The remainder of this document describes the system as it now stands: what each part does, and why it is arranged that way. Everything below is in place and verified.

1. From events to something a GPU can read

Each dataset arrives as raw events — one row per event, giving its pixel coordinates, its timestamp in microseconds, and whether the light went up or down. A graphics processor cannot work on a ragged stream of this kind; it needs regular blocks. The pipeline therefore filters sensor noise, divides each recording into a fixed number of equal time bins, and counts the events falling in each bin, producing a tensor of shape [time, batch, polarity, height, width].

This packing step is a concession to the hardware, not a property of the method, and it is precisely why the synaptic-operation estimate earns its place beside the measured energy. Every model in the experiment — spiking and conventional alike — receives exactly this tensor, from exactly the same cache.

The cache is content-addressed: its identity is a hash of the binning settings and the noise filter, so a change to either produces a new cache rather than silently reusing frames built under different settings. Storage tier — memory, disk, hybrid or GPU — is chosen automatically from the machine's free resources, and preprocessing is paid once per sample rather than once per epoch.

2. One network definition, five models

The architecture is described entirely in configuration as an ordered list of convolutional blocks plus an optional hidden fully-connected layer. A single builder reads that list and assembles the network: convolution, neuron, pooling, repeated per block, then flatten, then the readout. Every convolution, pooling and linear layer is standard PyTorch and therefore literally the same code in all five models.

Only the neuron is injected. Each of the four spiking frameworks supplies its own leaky-integrate-and-fire implementation through a common adapter, and the non-spiking control supplies a ReLU unit through the same interface. Because the network is shared rather than written out five times, “identical architecture, only the neuron changed” is true by construction rather than by careful copying.

Model	Neuron supplied	Role
snnTorch	<code>snn.Leaky</code>	spiking framework under test
Norse	<code>LIFBoxCell</code>	spiking framework under test
SpikingJelly	<code>LIFNode</code>	spiking framework under test
Sinabs	<code>sinabs.layers.LIF</code>	spiking framework under test
Control	<code>ReLU</code>	the denominator every spiking result is measured against

The control is not a frame-camera network. It reads the same event frames from the same cache, is built by the same builder from the same block list, and trains with the same optimiser and loss over the same number of timesteps. Its only difference is that its units pass a continuous value forward instead of accumulating a membrane potential and emitting a spike — which also means it carries no memory between timesteps. That single difference is the quantity the experiment measures.

3. Fairness, established before any run

All four spiking frameworks are configured from one neuron specification, written once and translated into each framework's own parameters, with the conversion arithmetic stated so it can be checked by reading. No framework default is ever relied upon: a missing or misspelled setting stops the run rather than falling back to a value buried in code, because every framework's default is a different neuron.

Two pre-flight checks run without touching a dataset or a GPU. The first builds the network for every framework and confirms that each layer's output shape is what the configuration implies, that the flattened width was measured rather than assumed, that each framework built the neuron it was asked for, and that all four begin from byte-identical weights. The second compares membrane and spike trajectories across the four, so that any later difference in behaviour is known to come from the implementation rather than from the configuration.

4. One architecture, on every application

The same network runs on every application: two convolutional blocks, each followed by a spiking layer and a pooling step, then the readout. Only the input shape and the output width follow the dataset. Within an application all five models run the identical network, so the comparison measures neurons rather than architectures.

Application	Network	Flattened width	Parameters
N-Caltech101	12 filters 5×5, 32 filters 5×5, two hidden layers of 10, output 101	76,608	777,643
DVS128 Gesture	the same, output 11	26,912	306,375
DAVIS Camera Pose	the same, continuous output of 7	76,608	546,595
DSEC optical flow	Encoder–decoder: 4 strided spiking stages 32–64–128–256 down, 4 transposed stages up with skip connections, 2 channels out at full resolution	not flattened	1,274,658

A normalisation layer sits between each convolution and its spiking layer. It is plain library code — the same module in all four frameworks, like the convolution and pooling around it — and it leaves the neuron untouched, so it changes nothing about what is compared. What it changes is whether the signal survives: it raises the readout's firing rate from 0.02% to 5.1%.

Two readouts are non-spiking by necessity. Pose is seven signed real numbers and flow is two per pixel; a spike count cannot represent either, so those networks end in a plain linear or convolutional layer. The encoder — where all the computation and all the sparsity lies — remains fully spiking, so the quantities being measured are unaffected.

5. Why the network ends where it does, with no hidden layer

This section records why depth was not added between the convolutional stack and the readout. Every figure is measured on N-Caltech101 at 180×240, T=16, on real cached event frames, and is reproducible with the architecture probe described in section 8.

The division of labour

The convolutional blocks and the hidden layers are not doing the same job, and that is what makes the placement decision straightforward.

The filters extract. The two convolutional blocks are where spatial structure is found — edges, corners, the shapes an object leaves behind as the camera sweeps across it. They operate on the grid, where position still means something. Their measured activity is healthy and rising: 5.9% at the first spiking layer, 11.8% at the second. They are doing their work.

The hidden layers compose. Once the grid is flattened, position is gone and what remains is a list of extracted features. Combining those features into a decision is a different operation from extracting them, and it is the one the original architecture had no room for at all — it went straight from flatten to output, one linear map.

The input width is not a free parameter in any of this. It follows from the sensor size, the filter sizes and the pooling, and cannot be set directly. The output width is fixed by the task. **The hidden layers are the only place in the network where depth can be chosen rather than derived.**

A narrow spiking layer is a rank limit, not a summary

A narrow fully-connected layer was tried between the two, on the reasoning that it would let the network compose features rather than read them off directly. Measurement refuted it, for a reason specific to spiking layers.

A conventional narrow layer of ten units carries ten continuous numbers — ten usable dimensions. A spiking one carries binary pulses at a low rate: measured, about 0.89 of its ten units fire in any timestep, which is effectively one-hot. The vector the readout integrates is then confined to a handful of directions, however many samples pass through. Across 128 real samples, only 5 genuinely different messages reached the readout — for a task needing 101.

Separating a hundred and one classes from five distinguishable messages is not a training difficulty that more epochs would solve; it is arithmetically impossible. A full five-epoch run confirmed it — the training loss flatlined at 4.29 against a chance level of 4.615, and accuracy stalled at 7.8%, below what predicting the single largest class would score.

hidden layers	parameters	memory per sample	units firing per step	usable directions
[10, 10]	777,643	172.3 MB	0.89	5
[128, 64]	9,831,105	172.4 MB	0.22	26
[256, 128]	19,668,161	172.4 MB	0.51	53
none — as built	7,747,841	172.3 MB	4.59	83

Measured on 128 real samples. “Usable directions” counts the singular values of the sample-by-unit matrix above 1% of the largest — a stable summary of how many genuinely different messages reach the decision, where an exact rank with a hard cutoff is not.

Narrowing is not rescued cheaply by widening. A wider spiking layer also fires more sparsely, so the count climbs far more slowly than the width: 256 units reach 53 usable directions at nineteen million parameters, against 83 for no hidden layer at all at under eight million. A single much wider layer — four to five times the class count — does saturate the measure, but at four times the parameters of the direct readout, and stacking two such layers collapses firing again, from 19.5 units per step to 1.3. What composing features gains is lost by passing them through a second sparse spiking bottleneck.

The decisive objection is fairness rather than accuracy. The same bottleneck costs the spiking model about twice what it costs the control -- 5 usable directions against 9 -- because ReLU units pass continuous values while spikes are binary and sparse. A structure that handicaps one side of the comparison for reasons unrelated to the question being asked has no place in it, whatever it might do for either model alone.

So the network ends as it began: two convolutional blocks, then the readout. Depth was tried where it looked free, measured, and removed.

Convolutional depth is the expensive direction

The other way to add capacity is another convolutional block. That costs 14% more memory per sample — 196.2 MB against 172.3 MB — and memory is what fixes the batch size, which is the single largest influence on gradient quality in an experiment running one seed with no replicates. It also buys capacity in the part of the network that was already working: activity through the two blocks is healthy and rising, 5.9% at the first spiking layer and 11.8% at the second. Nothing measured suggests feature extraction is where this network is short.

Why DSEC is arranged differently

DSEC has no hidden layers, and it is the deepest network in the set: nine learned layers against five. Its depth is in an encoder–decoder — four strided spiking stages compressing 640×480 down to a 30×40 grid of 256 features, then four transposed stages rebuilding full resolution, with skip connections carrying spatial detail across.

A fully-connected hidden layer cannot go there, because it would require flattening, and flattening discards which pixel is which. For a label or a pose that is harmless; for optical flow it is fatal. The output is 614,400 numbers — a motion vector at every pixel. Through ten neurons, every one of those values becomes a fixed combination of the same ten numbers, so the predicted flow field could only ever be ten patterns scaled and added together. A car moving left while a cyclist moves right is not expressible.

Same principle, different application of it: depth goes where the task allows it, and nowhere else.

In one sentence. The filters were already doing their job; the flattened width and the output width are both derived rather than chosen; hidden layers are the only free place left to add composition, and they are genuinely free — so two were added, which is the minimum that composes anything, and nothing beyond two was added because nothing measured asks for it.

6. Measurement

Instrumentation is built into the run rather than bolted on afterwards. A background monitor samples processor and GPU utilisation, power draw, memory, temperature and clock speeds throughout, and establishes an idle power baseline so that energy can be reported both as a total and as the dynamic portion attributable to the work itself.

Three measurements are taken as their own separate passes, deliberately outside any timed region, because each answers a different question and each would distort the others if folded in: single-sample latency, which is what a real-time deadline depends on; activation density, which requires per-layer counters that cost time to maintain; and adversarial robustness, which attacks the trained model across a range of perturbation budgets and reports how far accuracy falls.

Activation density deserves a note, because it is the measure that makes the spiking and non-spiking models comparable at all. It is gathered from counters inside each layer that record whether a unit emitted anything, not what it emitted. A spiking layer records its binary spikes; a conventional layer records whether its output exceeded zero. Both therefore accumulate the same quantity, and the sparsity claim becomes something measured rather than asserted.

7. Records

Results are written outside the code repository, so that they survive any change to the code that produced them. Each application has its own directory; each run within it has its own folder named by a run identifier that combines timestamp, model and seed.

File	Contents	Behaviour
runs.csv	One row per run: sixty-nine columns covering identity, configuration, architecture, task score, speed, activation density, internal dynamics, memory, energy, and the machine it ran on.	Append-only across every model and application
epochs.csv	One row per training epoch.	Append-only
layers.csv	One row per layer: neuron count, total activations, opportunities, density.	Append-only
<run_id>/	That run's own training and test logs, adversarial-robustness results, diagnostic figures, and optionally the trained model.	One folder per run

Every row carries the run identifier, a hash of the fully resolved configuration, the model name, the neuron type, the architecture description and a fingerprint of the initial weights. Any figure can therefore be traced back to the row that describes it and the configuration that produced it, and a configuration edited after the fact cannot quietly reattach itself to old results. A metric that could not be measured on a given machine is written as an empty cell rather than a missing column, so every row stays readable and comparable.

8. How a run is executed

One command per application. Each runs the four spiking frameworks and then the control, one after another, unattended, writing a log per model. A model that fails does not stop the ones after it; the outcome of each is reported in a summary at the end.

```
bash scripts/run_n_caltech101.sh
bash scripts/run_dvs128_gesture.sh
bash scripts/run_davis_pose.sh
bash scripts/run_dsec.sh
```

A fifth script, `run_architecture_probe.sh`, records the architecture evidence described in section 4. It trains nothing — every figure comes from a built network and a handful of passes — so the whole argument re-runs in a couple of minutes per application, on any machine, and writes to its own folder outside the comparison matrix.

Four applications times five models gives **twenty runs**. Every run uses a single fixed seed, which is also what makes all five models start from identical weights. Adversarial evaluation is active throughout; adversarial *training* is deliberately switched off in all twenty, because enabling it would mean comparing differently-trained models rather than differently-implemented ones. It is reserved for a later phase, applied to whichever framework the first twenty runs identify as strongest.

9. What the configuration guarantees

- Every setting is stated explicitly. A missing or misspelled key stops the run and names the closest valid alternative, rather than falling back to a default and reporting success.
- An experiment file may only override settings that already exist in the base configuration, so a typo cannot silently become a new and unread setting.
- Input shape and class count come from the dataset registry, never from the architecture file, so they cannot be set to something the data does not support.
- The flattened width between the convolutional stack and the classifier is computed by formula and independently measured by a real forward pass; the two must agree.
- One optimiser and one loss serve every model, because giving a framework its own would compare training recipes rather than frameworks.
- Batch size is calibrated against live GPU memory and recorded with the results, since it is the single largest influence on wall-clock time and two runs are only comparable on speed once it is known to match.

10. First results — DVS128 Gesture

The first of the four applications has been run from end to end. All five models — the four spiking frameworks and the non-spiking control — trained on DVS128 Gesture on 17 September 2026, one after another from a single command, on one NVIDIA RTX 5060 laptop GPU. Each ran for the same fifteen epochs over the same sixteen time bins with the same optimiser and the same single seed, and each started from byte-identical weights: all five records carry the same configuration hash, the same weight fingerprint and the same 306,287 trainable parameters. What follows is therefore a difference between neurons, not between networks.

Model	Score	Units that fired	Energy for one answer	Training	One answer
snnTorch	73.5%	2.2%	97 μ J	133 s	56 ms
Norse	72.3%	2.4%	110 μ J	137 s	47 ms
SpikingJelly	75.8%	2.6%	121 μ J	99 s	20 ms
Sinabs	72.7%	2.2%	96 μ J	104 s	45 ms
Control (ReLU)	69.3%	14.4%	436 μ J	88 s	20 ms

Score is accuracy on the held-out test recordings. *Units that fired* is the fraction of units across the network that emitted anything during the test pass — spikes for the spiking models, a value above zero for the control. *Energy for one answer* is the synaptic-operation estimate rather than a measurement: the operations one inference genuinely needs at the measured activity, priced at a flat cost each. *One answer* is the median time to answer a single sample, taken in its own untimed pass. Batch size was calibrated against free memory at the start of each run and is reported in the second table, because the wall-clock figures cannot be read without it.

The prediction held, on the application where it was meant to. Gesture is the time-dependent end of the axis: the label is a movement, and no single moment distinguishes a clockwise circle from a counter-clockwise one. All four spiking models scored above the control, by three to six and a half points. The control is not a weak opponent put up to lose — it reads the same frames from the same cache, the same builder assembles it, and it trains with the same optimiser and loss over the same sixteen timesteps. Its one difference is that it carries nothing from one timestep to the next, and on this task that is what it costs.

The sparsity is measured rather than asserted. Across the whole network the spiking models left between 97.4% and 97.8% of their units silent on any given pass; the control left 85.6%. Roughly six times as much of the conventional network is busy for every answer it produces.

On the hardware this work is aimed at, that sparsity is worth about four times. Counting the operations each inference genuinely requires at its measured activity, one answer costs 96 to 121 microjoules from a spiking model against 436 from the control. The estimate is deliberately conservative: it charges a spiking operation and a conventional one the same price, so it credits only the work avoided and none of the cheaper arithmetic a neuromorphic chip would also bring.

On the hardware it actually ran on, the sparsity is worth nothing at all. The control finished training first and drew the least energy doing it. A graphics processor computes a dense block of numbers whether or not most of them are zero, so silence buys no time there, and the spiking models pay for their memory of the past in seconds and joules. This is the expected outcome, and precisely why the two energy figures are reported side by side and never merged.

Model	GPU energy, whole training run	Above idle	Peak memory	Batch size
snnTorch	7.3 kJ	5.3 kJ	4.0 GB	30
Norse	8.9 kJ	7.8 kJ	3.9 GB	36
SpikingJelly	5.3 kJ	3.9 kJ	3.9 GB	36
Sinabs	5.9 kJ	4.5 kJ	3.7 GB	44
Control (ReLU)	3.2 kJ	1.9 kJ	4.3 GB	86

Energy drawn over the whole training run, and the portion of it above the idle baseline measured on the same machine immediately beforehand. Batch size is calibrated per run against the memory free at that moment, so the models did not all see the same number of samples per step: the control fits more than twice as many as any spiking model, which is part of why it finishes sooner.

The four frameworks differ more in speed than in score. Their accuracies span 3.4 points, which one seed with no repeats cannot separate from noise — nothing here ranks one framework above another on quality. Their speeds are another matter. SpikingJelly trained in 99 seconds where Norse took 137, and answered a single sample in 20 milliseconds where snnTorch took 56: the same neuron on paper, the same starting weights, nearly three times the delay. That gap is implementation, and it is the kind of difference four independent frameworks were included to expose.

Model	First layer	Second layer	Readout
snnTorch	4.0%	0.5%	9.8%
Norse	4.3%	0.5%	9.7%
SpikingJelly	4.7%	0.5%	8.3%
Sinabs	4.0%	0.4%	8.4%
Control (ReLU)	23.1%	5.7%	20.7%

Activity layer by layer, from the per-layer counters. The pattern is the same in all four spiking models and differs from the control in both places: the first layer is where the events arrive and is the busiest, the second is nearly silent, and the readout fires often because it has only eleven units and one of them has to win.

What this does not yet show. One application, one seed, no repeats. Every spiking model reached essentially perfect accuracy on the training recordings by the fifteenth epoch while the control stalled near 91%, so all five are fitting a small dataset closely, and the test scores sit below what larger published networks reach on this task — the cost, accepted in advance, of one deliberately small architecture shared across every application. The claim the four applications test together is that the advantage widens as dependence on time increases, and that needs the near-timeless end of the axis before it can be said to hold. Gesture on its own shows only that the advantage is present, and sizeable, where it was predicted to be largest.